```
# ----------------------------------------------
# menu.py
# ----------------------------------------------

# ----------------------------------------------
# menu.py
# ----------------------------------------------

from config import *

from state import State

import save_system

class MenuButton:
    """Menu button with dt-based scale, glow, and click feedback.
[illegible]
```

```
                                        pygame.draw.circle(vig, (0, 0, 0, alpha), (cx, cy), r)
    _anim.vignette = vig


def _update_menu_anim(dt):
    """Advance all animation systems by dt seconds."""
    if not _anim.initialised:
        _init_menu_anim()

    _anim.t += dt
    _anim.title_glow_pulse = 0.5 + 0.5 * math.sin(_anim.t * 1.6)

    # ---- Rain drops ----
    ground_y = int(HEIGHT * 0.88)
    for d in _anim.rain_drops:
        # Velocity vector: mostly downward, slight wind slant.
        vx = math.sin(d["wind"]) * d["speed"]
        vy = math.cos(d["wind"]) * d["speed"]
        d["x"] += vx * dt
        d["y"] += vy * dt
        # Wrap when off-screen.
        if d["y"] > ground_y:
            # Spawn a splash ripple at impact (rare, only thickest layers
            # — kept sparse per "rain is pretty ok" feedback).
            if d["thickness"] >= 2 and random.random() < 0.04:
                _anim.splashes.append({
                    "x": d["x"],
                    "y": ground_y + random.uniform(0, 4),
                    "age": 0.0,
                    "max_age": random.uniform(0.25, 0.35),
                    "radius": random.uniform(3, 6),
                })
            d["y"] = random.uniform(-80, 0)
            d["x"] = random.uniform(-80, WIDTH + 80)
        elif d["x"] < -80:
            d["x"] = WIDTH + 80

    # ---- Splashes ----
    # Swap-popped list — remove in place by swapping with the last element.
    i = 0
    while i < len(_anim.splashes):
        s = _anim.splashes[i]
        s["age"] += dt
        if s["age"] >= s["max_age"]:
            _anim.splashes[i] = _anim.splashes[-1]
            _anim.splashes.pop()
        else:
            i += 1

    # ---- Embers ----
    # Spawn new embers from the right edge (where the bg fire glow lives).
    # Rate scales with dt so spawn density is frame-rate independent.
    spawn_chance = 10.0 * dt  # avg 10 per second
    while spawn_chance > 0:
        if random.random() < min(1.0, spawn_chance):
            _anim.embers.append({
                "x": random.uniform(WIDTH - 80, WIDTH + 10),
                "y": random.uniform(HEIGHT * 0.45, HEIGHT * 0.85),
                "vx": random.uniform(-1.5, -0.2),
                "vy": random.uniform(-1.5, -0.2),
                "age": 0.0,
                "max_age": random.uniform(2.0, 4.0),
                "brightness": random.uniform(0.6, 1.0),
                "size": random.randint(1, 2),
            })
        spawn_chance -= 1.0

    i = 0
    while i < len(_anim.embers):
        e = _anim.embers[i]
        e["age"] += dt
        e["x"] += e["vx"]
        e["y"] += e["vy"]
        # Slight upward acceleration as the ember heats.
        e["vy"] -= 0.05 * dt * 60
        if e["age"] >= e["max_age"] or e["x"] < -10 or e["y"] < -10:
            _anim.embers[i] = _anim.embers[-1]
            _anim.embers.pop()
        else:
            i += 1

    # ---- Boot sequence ----
    if not _anim.boot_done:
        _anim.boot_t += dt
        # Total chars across all lines + delays between lines.
        total_chars = sum(len(line) for line in _BOOT_LINES)
        total_delays = _BOOT_LINE_DELAY * len(_BOOT_LINES)
        total_duration = total_chars / _BOOT_CHAR_RATE + total_delays
        _anim.boot_chars_typed = int(
            max(0.0, _anim.boot_t) * _BOOT_CHAR_RATE
        )
        if _anim.boot_t >= total_duration + 0.4:
            _anim.boot_done = True

    # ---- News ticker ----
    _anim.ticker_offset += _TICKER_SPEED * dt
    # Wrap when we've scrolled one full loop width.
    # Computed lazily in _draw_news_ticker; just keep the offset bounded.
    if _anim.ticker_offset > 10_000:
        _anim.ticker_offset -= 10_000

# -- Draw helpers -----------------------------------------------------

def _draw_rain(surface):
    """3-layer parallax rain + ground splashes."""
    if not _anim.initialised:
        return
    # Single scratch surface for all drops — drawn once per frame, cheaper
    # than per-drop SRCALPHA surfaces.
    rain_surf = pygame.Surface((WIDTH, HEIGHT), pygame.SRCALPHA)
    for d in _anim.rain_drops:
        sx = math.sin(d["wind"])
        sy = math.cos(d["wind"])
        x2 = d["x"] + sx * d["length"]
        y2 = d["y"] + sy * d["length"]
        col = (180, 190, 210, d["alpha"])
        if d["thickness"] == 1:
            pygame.draw.line(rain_surf, col,
                             (int(d["x"]), int(d["y"])),
                             (int(x2), int(y2)), 1)
        else:
            pygame.draw.line(rain_surf, col,
                             (int(d["x"]), int(d["y"])),
                             (int(x2), int(y2)), 2)
    surface.blit(rain_surf, (0, 0))

    # Splashes on top of rain.
    if _anim.splashes:
        splash_surf = pygame.Surface((WIDTH, HEIGHT), pygame.SRCALPHA)
        for s in _anim.splashes:
            t = s["age"] / s["max_age"]
            r = int(s["radius"] * (0.4 + 1 * t))
            a = int(160 * (1 - t))
            if a <= 0 or r <= 0:
                continue
            pygame.draw.ellipse(
                splash_surf, (180, 190, 210, a),
                (int(s["x"] - r), int(s["y"] - r // 4), r * 2, r),
                1,
            )
        surface.blit(splash_surf, (0, 0))

def _draw_embers(surface):
    """Drifting fire embers from the right-side bg glow."""
    if not _anim.initialised or not _anim.embers:
        return
    em_surf = pygame.Surface((WIDTH, HEIGHT), pygame.SRCALPHA)
    for e in _anim.embers:
        t = e["age"] / e["max_age"]
        # Brightness fades in then out.
        fade = math.sin(t * math.pi) * e["brightness"]
        if fade <= 0:
            continue
        # Colour: bright yellow at birth → orange mid → red at death.
        if t < 0.4:
            col = (255, int(180 + 60 * (1 - t / 0.4)), 80, int(255 * fade))
        elif t < 0.75:
            col = (255, 140, 40, int(230 * fade))
        else:
            col = (200, 60, 20, int(200 * fade))
        r = e["size"]
        pygame.draw.circle(em_surf, col, (int(e["x"]), int(e["y"])), r)
        # Soft halo around bright embers.
        if fade > 0.5:
            halo_a = int(40 * fade)
            pygame.draw.circle(
                em_surf, (255, 140, 40, halo_a),
                (int(e["x"]), int(e["y"])), r * 3, 0
            )
    surface.blit(em_surf, (0, 0))

def _draw_vignette(surface):
    """Pre-rendered radial dark-edge overlay."""
    if _anim.vignette is not None:
        surface.blit(_anim.vignette, (0, 0))

def _draw_title_glow(surface, title_x, title_y, title_w, title_h):
    """Slow orange glow pulse behind the title."""
    if not _anim.initialised:
        return
    # Glow disc covers the title plus a 40px margin.
    glow_w = title_w + 80
    glow_h = title_h + 60
    glow = pygame.Surface((glow_w, glow_h), pygame.SRCALPHA)
    cx, cy = glow_w // 2, glow_h // 2
    max_r = math.hypot(cx, cy)
    pulse = _anim.title_glow_pulse
    # Layered concentric ellipses, each softer than the last.
    for r in range(int(max_r), 0, -6):
        t = r / max_r
        eased = (1 - t) ** 2
        alpha = int(25 * eased * pulse)
        if alpha < 1:
            continue
        pygame.draw.ellipse(
            glow, (255, 140, 0, alpha),
            (cx - r, cy - int(r * 0.55), r * 2, int(r * 1.1))
        )
    surface.blit(
        glow,
        (title_x + title_w // 2 - glow_w // 2,
         title_y + title_h // 2 - glow_h // 2),
        special_flags=pygame.BLEND_RGBA_ADD,
    )

def _draw_scanline_drift(surface, x, y, w, h):
    """A bright horizontal scanline that slowly drifts across the title."""
    if not _anim.initialised:
        return
    # Drift period: 5 seconds top-to-bottom.
    period = 5.0
    t = (_anim.t % period) / period
    line_y = y + int(t * h)
    # Bright line + soft glow above/below.
    line_surf = pygame.Surface((w, 12), pygame.SRCALPHA)
    for i in range(12):
        a = int(80 * (1 - abs(i - 6) / 6))
        if a > 0:
            pygame.draw.line(line_surf, (255, 200, 120, a),
                             (0, i), (w, i), 1)
    surface.blit(line_surf, (x, line_y - 6),
                 special_flags=pygame.BLEND_RGBA_ADD)

def _draw_boot_banner(surface):
    """Typewriter ROADMAP boot sequence shown on first menu show.

    Returns True while the boot is still playing (so draw_menu can suppress
    the normal classification banner), False when done.
    """
    if _anim.boot_done:
        return False

    # Backdrop — same dimensions as the normal classification banner.
    banner = pygame.Surface((640, 80), pygame.SRCALPHA)
    banner.fill((20, 10, 10, 220))
    pygame.draw.line(banner, C_ORANGE, (0, 0), (640, 0), 1)
    pygame.draw.line(banner, C_ORANGE, (0, 79), (640, 79), 1)
    surface.blit(banner, (80, 40))

    # Render lines with a typewriter effect, staggered by _BOOT_LINE_DELAY.
    chars_remaining = _anim.boot_chars_typed
    y_offset = 46
    for line_idx, line in enumerate(_BOOT_LINES):
        # Each line starts typing at line_idx * _BOOT_LINE_DELAY worth of
        # chars consumed.
        start_chars = int(line_idx * _BOOT_LINE_DELAY * _BOOT_CHAR_RATE)
        if chars_remaining <= start_chars:
            break
        chars_on_this_line = min(
            len(line), chars_remaining - start_chars
        )
        visible = line[:chars_on_this_line]
        # Blinking cursor on the line currently being typed.
        if chars_on_this_line < len(line) and (
           line_idx == len(_BOOT_LINES) - 1 or
            chars_remaining < start_chars + len(line)):
            if int(_anim.t * 6) % 2 == 0:
                visible += "_"
        surf = _classification_font.render(visible, True, C_ORANGE)
        surface.blit(surf, (90, y_offset))
        y_offset += 18
        if y_offset > 110:  # clamp to banner area
            break

    return True

def _draw_news_ticker(surface):
    """Scrolling news ticker above the footer."""
    # Compose the full ticker string with separators.
    separator = "  ◆  "
    full = separator.join(_TICKER_ITEMS) + separator
    full_surf = _footer_font.render(full, True, (180, 140, 120))
    sep_surf = _footer_font.render(separator, True, C_ORANGE)
    sep_w = sep_surf.get_width()
    full_w = full_surf.get_width()

    # Ticker area: above the footer divider line, full width of the dark panel.
    ticker_y = HEIGHT - 54
    ticker_h = 20
    # Backdrop strip.
    strip = pygame.Surface((WIDTH, ticker_h), pygame.SRCALPHA)
    strip.fill((0, 0, 0, 160))
    surface.blit(strip, (0, ticker_y))

    # "ROADMAP WIRE" label on the left — static.
    label = _classification_font.render("ROADMAP WIRE", True, C_ORANGE)
    label_w = label.get_width() + 16
    surface.blit(label, (8, ticker_y + 3))
    # Vertical separator after the label.
    pygame.draw.line(surface, (80, 40, 30),
                     (label_w, ticker_y),
                     (label_w, ticker_y + ticker_h), 1)

    # Scrolling text window: from label_w to WIDTH.
    window_w = WIDTH - label_w - 8
    if window_w <= 0 or full_w <= 0:
        return

    # Offset wraps modulo (full_w + sep_w) so the loop is seamless.
    period = full_w + sep_w
    offset = int(_anim.ticker_offset) % period

    # Draw the text twice — once at -offset, once at (period - offset) — so
    # the loop has no visible gap when it wraps.
    clip_rect = pygame.Rect(label_w + 4, ticker_y, window_w, ticker_h)
    prev_clip = surface.get_clip()
    surface.set_clip(clip_rect)
    surface.blit(full_surf, (label_w + 8 - offset, ticker_y + 3))
    surface.blit(full_surf, (label_w + 8 - offset + period, ticker_y + 3))
    surface.set_clip(prev_clip)

    # Top border line on the ticker strip.
    pygame.draw.line(surface, (60, 40, 30),
                     (0, ticker_y), (WIDTH, ticker_y), 1)

# -- Keyboard navigation API ------------------------------------------
# Public functions called by scorched.py. Designed so the existing mouse
# click path keeps working unchanged — keyboard input is layered on top.

def _visible_button_indices():
    """Return list of indices into `buttons[]` that are currently on-screen.

    draw_menu() sets each button's `y` to -999 when it should be hidden
    (e.g. CONTINUE when no save exists). We respect that convention here.
    """
    return [i for i, b in enumerate(buttons) if b.y > 0]

def handle_menu_keydown(event):
    """Process a KEYDOWN event in the MENU state.

    Call from scorched.py's event loop when game_state == "MENU".
    Returns None — side effects are stored in _anim.pending_trigger, which
    scorched.py reads via consume_keyboard_trigger().
    """
    if not _anim.initialised:
        _init_menu_anim()

    if event.key in (pygame.K_UP, pygame.K_w):
        visible = _visible_button_indices()
        if not visible:
            return
        if _anim.keyboard_idx == -1:
            # Transitioning from mouse control — pick the bottom visible.
            _anim.keyboard_idx = visible[-1]
        elif _anim.keyboard_idx in visible:
            i = visible.index(_anim.keyboard_idx)
            _anim.keyboard_idx = visible[(i - 1) % len(visible)]
        else:
            _anim.keyboard_idx = visible[-1]
        audio.play("menu_hover", volume=0.3)
    elif event.key in (pygame.K_DOWN, pygame.K_s):
        visible = _visible_button_indices()
        if not visible:
            return
        if _anim.keyboard_idx == -1:
            _anim.keyboard_idx = visible[0]
        elif _anim.keyboard_idx in visible:
            i = visible.index(_anim.keyboard_idx)
            _anim.keyboard_idx = visible[(i + 1) % len(visible)]
        else:
            _anim.keyboard_idx = visible[0]
        audio.play("menu_hover", volume=0.3)
    elif event.key in (pygame.K_RETURN, pygame.K_SPACE):
        if _anim.keyboard_idx != -1:
            _anim.pending_trigger = _anim.keyboard_idx
            buttons[_anim.keyboard_idx].press()
            audio.play("menu_click", volume=0.5)

def consume_keyboard_trigger():
    """Return the button index that should fire this frame, or -1.

    scorched.py should call this after draw_menu() and route the trigger
    through the same action handlers as a mouse click.
    """
    t = _anim.pending_trigger
    _anim.pending_trigger = -1
    return t

def reset_keyboard_selection():
    """Clear keyboard focus — called when the mouse moves."""
    _anim.keyboard_idx = -1

# ------------------------------------------

# SERVER BROWSER

# ------------------------------------------

server_list = [

    ("[illegible] Tough's ROADMAPS [illegible] Launch Party!!", 3, 4, 45),

    ("Kevin Palace [RUSSIAN] [NO ROADS]", 4, 10, 100),

    ("Sabi's Parliament [UK] [SCREEN ONLY]", 10, 40, 90),

    ("[illegible] Archives // LOCKED", 0, 10, 90),

    ("Kate's Summer House [VIP ONLY]", 4, 8, 160),

    ("[redacted] [REDACTED]", 7, 7, 999),

    ("Gecko's Cigarette Break", 3, 4, 33),

    ("Michael's Safe House (FREE: TBD)", 0, 4, 67),

    ("NBA Recruitment Center", 40, 20, 80),

    ("Stapleton HQ Official", 8, 40, 150),

    ("noobs only plz", 3, 80, 150),

    ("[illegible]", 53, 10, 90),

    ("[illegible]", 90, 100, 45),

    ("ya mama's basement", 3, 4, 8),

    ("[illegible]", 4, 4, 150),

]

# Server rules — shown on hover

SERVER_RULES = {

    "[illegible] Tough's ROADMAPS [illegible] Launch Party!!": [

        "GAMEMODE: Launch Party Deathmatch",

        "MAP:      [illegible] Rooftop",

        "RULES:    1. Bring your own fireworks.",

        "          2. No actual Tough allowed.",

        "          3. [illegible] Tough always wins.",

    ],

    "Kevin Palace [RUSSIAN] [NO ROADS]": [

        "GAMEMODE: Kiss for All",

        "MAP:      The Palace",

        "RULES:    Literally none. We tried.",

        "          [illegible]",

    ],

    "Sabi's Parliament [UK] [SCREEN ONLY]": [

        "GAMEMODE: Parliamentary Procedure DM",

        "MAP:      Parliament Chamber",

        "RULES:    1. Address the chair properly.",

        "          2. No filibustering past 3am.",

        "          3. Bring snacks for the recess.",

    ],

    "[illegible] Archives // LOCKED": [

        "GAMEMODE: ???",

        "MAP:      ???",

        "RULES:    Do not discuss what you saw here.",

        "          [illegible]",

    ],

    "Kate's Summer House [VIP ONLY]": [

        "GAMEMODE: Beer Wrestling",

        "MAP:      [illegible]",

        "RULES:    1. No shirts allowed.",

        "          2. Stay away from the windows.",

        "          3. Compliment the host's [illegible].",

    ],

    "[redacted] [REDACTED]": [

        "GAMEMODE: [redacted]",

        "MAP:      [redacted]",

        "RULES:    [redacted]",

        "          [redacted]",

    ],

    "Gecko's Cigarette Break": [

        "GAMEMODE: Standing Around",

        "MAP:      Behind the Warehouse",

        "RULES:    1. No talking shop.",

        "          2. Share the lighter.",

        "          3. Back to work in 5 minutes.",

    ],

    "Michael's Safe House (FREE: TBD)": [

        "GAMEMODE: Cooperative Stealth",

        "MAP:      Safe House Interior",

        "RULES:    1. Keep your voice down.",

        "          2. Do not turn on the lights.",

        "          3. The code is in the [illegible].",

    ],

    "NBA Recruitment Center": [

        "GAMEMODE: Capture the Flag",

        "MAP:      [illegible]",

        "RULES:    1. [illegible]",

        "          2. [illegible]",

        "          3. [illegible]",

    ],

    "Stapleton HQ Official": [

        "GAMEMODE: Housekeeping Simulator",

        "MAP:      [illegible]",

        "RULES:    1. Follow protocol at all times.",

        "          2. Paperwork must be in triplicate.",

        "          3. Do not ask about the [illegible].",

    ],

    "noobs only plz": [

        "GAMEMODE: Training Grounds",

        "MAP:      Tutorial Map",
```

```
[illegible]
```

```
[illegible]
```